

Ver el error de hoy y acordar qué construimos

Un agente que mapea el catálogo a Shopee, decidido paso a paso.

War Room · Alephee × Craftech × AWS · 1/10/2026

Ver el error de hoy y acordar qué construimos

Lo que vamos a hacer hoy.

Decidimos, construimos y medimos. Cada bloque cierra con una versión que funciona.

- 1 · PUNTO DE PARTIDA | El error de hoy, el alcance y el contrato
- 2 · DISEÑAR EL AGENTE | Tipo de aplicación, prompt, dónde corre, modelo, stack y criterio de éxito
- 3 · V1 · EL AGENTE RESPONDE | Una sola llamada al modelo, sin herramientas: qué resuelve solo y qué no
- 4 · V2 · HERRAMIENTAS | Las tablas de Alephee como herramientas que el agente consulta
- 5 · V3 · CONTROL | Validaciones en código, correcciones del equipo y caché por SKU
- 6 · LA PRUEBA | El proceso de hoy contra V1, V2 y V3, y el camino a producción

Ver el error de hoy y acordar qué construimos

De un producto de Alephee a una publicación de Shopee.

Caso real · SKU 88904447 · Correia dentada · catálogo GM Brasil (Mercado Libre Brasil) → Shopee Brasil

Entra (Alephee)	Sale (Shopee)	Quién lo resuelve
Categoría: Correias Dentadas	Distribuição e Correias	La tabla
Condición del ítem: Novo	Condition: New	La tabla y la lista del canal
Tiempo de garantía: 6 meses	Duração da Garantia: 6 Months	El agente
Código OEM: no viene	Faltante, con motivo (hoy: ABS Plastic, rechazado por Shopee)	El guardrail

Ver el error de hoy y acordar qué construimos

Hoy: dos llamadas, un merge y un presupuesto que se agota.

- Categoría: la tabla `reference_category` y, si no alcanza, el modelo elige
- Atributos: dos llamadas en paralelo (con y sin referencia) y un merge por código
- USD 350 por mes: cuando se agota, se publica sin atributos

Ver el error de hoy y acordar qué construimos

Lo que Shopee rechazó y lo que aceptó mal.

Datos reales · WarRoom.zip del 24/09 · 30 productos

Caso	Qué salió	Qué pasó
88904447	Código OEM = ABS Plastic	Rechazado: el valor no pertenece al atributo
93221445	Inmetro Certification con varios valores	Rechazado: solo admite un valor
9 de 30	Valor -1 publicado como valor	Aceptado por Shopee
98550368	14 de 15 URN sin sufijo del canal	Aceptado por Shopee

Ver el error de hoy y acordar qué construimos

¿Qué hace y qué no hace?

DECISIÓN 1

¿Qué alcance tiene el agente del war room?

A · Un canal (Shopee) y una familia de productos: categoría + atributos

B · Todos los canales conectados (Shopee, Magalu, Tienda Nube...)

C · También título, descripción y marca

PROPUESTA · A. Un canal y una familia alcanzan para decidir el método y medirlo. Marca, título y descripción son redacción libre y no mapeo, así que quedan fuera. Los otros canales repiten el mismo diseño con sus propias tablas.

decisiones/01-alcance.md

Ver el error de hoy y acordar qué construimos

Entra un producto. Sale una propuesta de publicación.

- Entrada: el producto de Alephee (SKU, nombre, descripción, categoría legacy, atributos)
- Salida: la categoría de Shopee más los atributos con URN, valueId y valor
- Y dos listas más: missing (obligatorios sin dato) y rejected (atributos descartados, con motivo)

Ver el error de hoy y acordar qué construimos

El contrato, en código.

core/src/catalog/models.py · líneas 21–37

```
21 class MissingAttribute(_Strict):
22     urn: str = Field(description="URN of the mandatory channel attribute (or 'category').")
23     reason: str
24
25
26 class RejectedAttribute(_Strict):
27     legacyId: str = Field(description="Product attribute id, without the 'urn:attribute:' prefix.")
28     reason: str
29
30
31 class Listing(_Strict):
32     """Result of mapping one product to a channel listing."""
33
34     category: str | None = Field(description="Channel category URN, copied exactly. Null if it cannot be resolved.")
35     attributes: list[MappedAttribute]
36     missing: list[MissingAttribute] = Field(description="Mandatory channel attributes (or the category) that could not be filled.")
37     rejected: list[RejectedAttribute] = Field(description="Product attributes that are discarded, with the reason.")
```

Listing es lo que entrega la V1. Si el modelo devuelve algo que no cumple este esquema, el Workflow informa el error.

Ver el error de hoy y acordar qué construimos

¿Qué recibe y qué entrega?

DECISIÓN 2

¿Cómo se informa lo que no se pudo mapear?

A · No se informa: se publica lo que hay (hoy)

B · missing y rejected con motivo, en la misma salida

C · Un archivo de log aparte que alguien revisa

PROPUESTA · B. El faltante viaja con la publicación, y quien revisa ve el producto, el atributo y el motivo juntos. Un log aparte se separa del dato y nadie lo mira. La decisión 12 define qué pasa después con esas listas.

decisiones/02-contrato.md

Tomar las decisiones que definen la V1

Diseñar antes de escribir.

Siete decisiones que definen la V1

Tomar las decisiones que definen la V1

Cuatro formas de usar un modelo.

- SINGLE PROMPT | Una llamada y una respuesta. Barato y fácil de evaluar, pero no consulta nada
- WORKFLOW | Pasos fijos en código, y el modelo participa en uno o dos. Es predecible
- AGENTE | El modelo decide qué herramienta pedir y cuándo entregar, con un límite de rondas
- MULTIAGENTE | Varios agentes con roles distintos. Solo cuando uno no alcanza

Tomar las decisiones que definen la V1

¿Single prompt o agente?

La V1 se construye con la opción A · si gana otra, vemos qué cambia

DECISIÓN 3

¿Con qué tipo de aplicación empezamos?

A · Una sola llamada estructurada (V1) y sumar herramientas solo cuando un fallo lo justifique

B · Agente con herramientas desde el inicio

C · Workflow determinista, sin modelo

PROPUESTA · A. La V1 muestra qué resuelve el modelo solo y qué no, y cada fallo justifica la capa siguiente. También es la buena práctica de LlamaIndex: si el modelo no tiene herramientas que elegir, no hace falta un agente. El FunctionAgent entra en la V2. C no alcanza, porque elegir el valor equivalente de una lista (por ejemplo, '1' es Sim) necesita interpretación.

decisiones/03-tipo-de-aplicacion.md

Tomar las decisiones que definen la V1

Un prompt tiene cinco partes.

01 Rol: quién es

02 Tarea: qué hace y con qué entrada

03 Reglas: qué nunca hace

04 Formato: cómo entrega

05 Cuando falta dato: qué hace

Tomar las decisiones que definen la V1

Buenas prácticas que cambian el resultado.

- Lo fijo primero y lo variable al final: así funciona la caché de prompt (F4, F7)
- Salida con esquema, en lugar de pedir JSON en el texto
- Decir qué hacer cuando no sabe, además de qué hacer

Tomar las decisiones que definen la V1

¿Cómo garantizamos el formato?

La V1 usa la opción B · si gana otra, vemos qué cambia

DECISIÓN 4

¿Cómo se garantiza que la salida cumpla el contrato?

A · Pedir JSON en el texto y parsearlo (hoy)

B · Salida estructurada con un modelo Pydantic: el modelo devuelve un Listing validado

C · Corregir la respuesta después con expresiones regulares

PROPUESTA · B. El formato deja de ser un pedido en el prompt y pasa a ser un contrato. En LlamaIndex es `as_structured_llm(Listing)`: el modelo responde por tool calling y la librería valida contra el esquema. Si la salida no cumple, el Workflow devuelve el error en lugar de publicar algo a medias. C arregla síntomas: si el modelo omite un campo, la expresión regular no lo inventa.

decisiones/04-salida-estructurada.md

Tomar las decisiones que definen la V1

Salida estructurada en LlamalIndex.

core/src/catalog/v1.py · líneas 69–77

```
69     @step
70     async def map(self, ev: ContextReady) -> MappingCompleted:
71         try:
72             response = await self.llm.as_structured_llm(Listing).achat(ev.messages)
73         except Exception as exc: # noqa: BLE001 — any failure becomes a reported error, except credentials
74             if is_auth_error(exc):
75                 raise
76             return MappingCompleted(listing=None, error=f"{type(exc).__name__}: {exc}")
77         return MappingCompleted(listing=response.raw)
```

Una sola llamada. response.raw ya es un Listing validado.

Tomar las decisiones que definen la V1

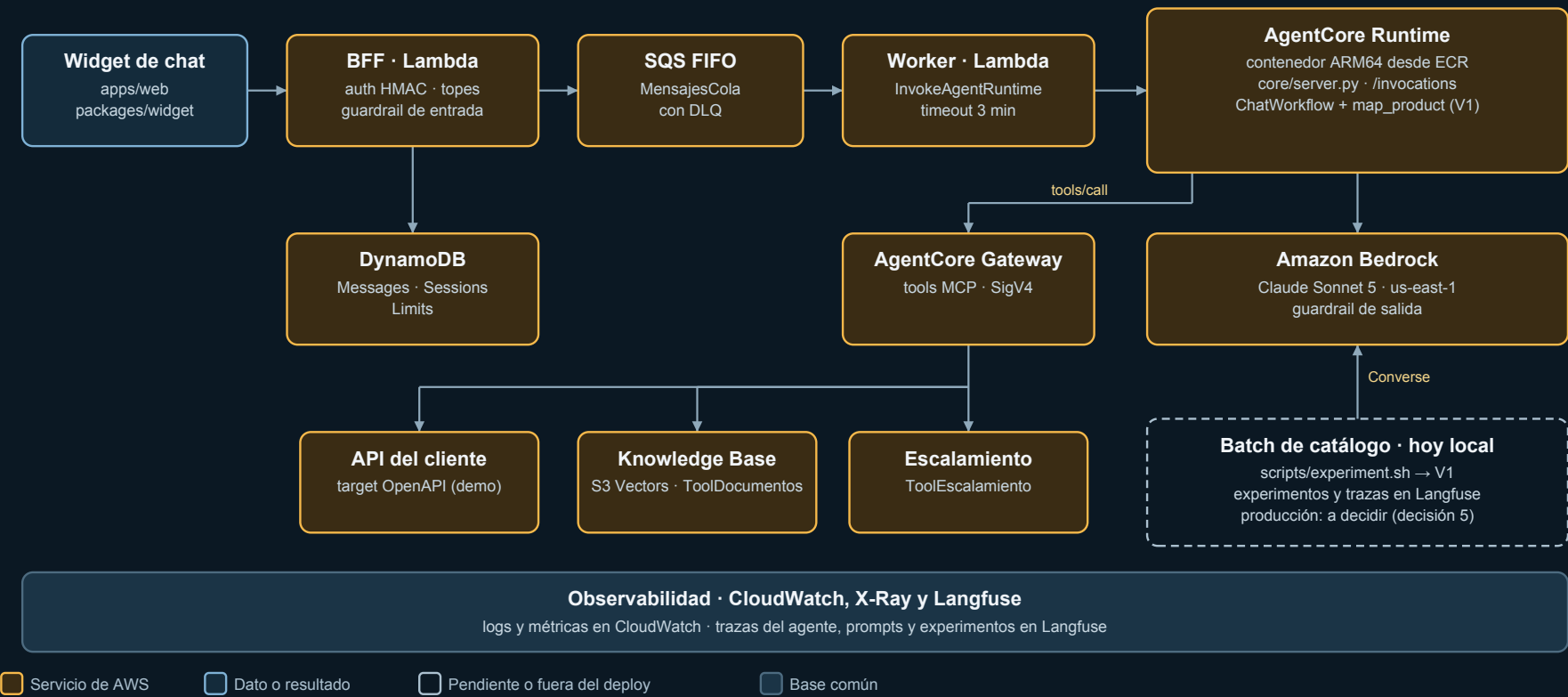
Dónde corre: AgentCore o contenedor propio.

Fuentes F1 a F3b en docs/warroom/fuentes.md

- AGENTCORE RESUELVE | Runtime sin servidores, identidad, memoria, gateway de herramientas y observabilidad. Son servicios separados que se pueden usar juntos o no (F2)
- DUDAS | No hay cifra publicada de arranque en frío. Las sesiones terminan a los 15 minutos sin actividad y duran 8 horas como máximo (F3b). Se cobra por CPU y memoria de la sesión (F3), y hay que confirmar la región
- ALTERNATIVA | Contenedor o Lambda propios: más control y más trabajo de operación. El agente es el mismo

Tomar las decisiones que definen la V1

La infraestructura, de punta a punta.



Es la infraestructura del template en infra/sst. El chat de la V1 se despliega en la cuenta sandbox con el stage warroom, y las trazas del agente van a Langfuse. AgentCore Memory se sacó del stack.

Tomar las decisiones que definen la V1

¿Dónde corre?

DECISIÓN 5

¿Dónde corre el agente?

A · Hoy en local y el chat de demo en AgentCore (stage warroom). El batch como proceso de Alephee se decide con el camino a producción

B · Todo en AgentCore desde el inicio

C · Todo en la infraestructura actual de Alephee

PROPUESTA · A. Hoy construimos y medimos en local. El chat va a AgentCore porque es su caso natural. Para el batch hay que medir sesiones, costo y arranque antes de decidir, y lo retomamos a las 16:15.

decisiones/05-donde-corre.md

Tomar las decisiones que definen la V1

Elegir el modelo: qué pesa en este caso.

- Seguir reglas sin inventar valores: es lo que más falla hoy
- Costo por token con caché de prompt: el presupuesto actual es de USD 350 por mes
- Latencia: no es una restricción (hoy el batch tarda 13 s por producto)
- Que esté disponible en la región y habilitado en la cuenta

Tomar las decisiones que definen la V1

Qué hay en Bedrock.

Fuente F6 · Models at a glance · verificado el 30/09/2026

Proveedor	Familias	Para este caso
Anthropic	Claude 5.x (Sonnet 5.5, Opus 5.5, Fable 5.1, Sonnet 5...), 4.x (Haiku 4.5...), 3.x	Candidato principal: reglas y tool calling. Sonnet 5 elegido el 25/09; 5.5 y Haiku 4.5 a comparar
Amazon	Nova 2 Lite, Nova Premier, Pro, Lite, Micro	Alternativa de costo a medir
OpenAI	GPT-5.x, GPT-6, GPT OSS	Continuidad con el proveedor actual, dentro de AWS
Meta · Mistral · DeepSeek · Qwen · otros	Llama 3.x y 4, Mistral Large 3, DeepSeek V3.2, Qwen3...	Abiertos; evaluar si el costo lo justifica

Tomar las decisiones que definen la V1

¿Qué modelo?

Tomada el 25/09 (CLAUDE.md, decisión 6) · hoy se valida o se cambia

DECISIÓN 6

¿Con qué modelo construimos y medimos?

A · Claude Sonnet 5 en Bedrock, vía Converse

B · Un modelo más chico y barato (Claude Haiku 4.5) y medir

C · Seguir con OpenAI y solo reordenar el prompt para la caché

PROPUESTA · A para construir hoy. Es la decisión del 25/09 y el ID está verificado en Converse. Claude Sonnet 5.5 también está en Bedrock (F6). Compararlo con Haiku 4.5 (B) es una corrida más del experimento con otra variable de entorno, y queda como pendiente con dueño. C mejora el costo pero no resuelve el contrato ni el control.

decisiones/06-modelo.md

Tomar las decisiones que definen la V1

El único módulo que sabe de Bedrock.

core/src/catalog/llm.py · líneas 8–20

```
8 # Converse accepts the `us.` and `global.` profiles; the bare model id has no on-demand throughput.
9 DEFAULT_MODEL_ID = "us.anthropic.claude-sonnet-5"
10 _AUTH_ERRORS = (NoCredentialsError, SSOError, TokenRetrievalError, UnauthorizedSSOTokenError)
11
12
13 def create_llm(env=os.environ) -> BedrockConverse:
14     return BedrockConverse(
15         model=env.get("MODEL_ID", DEFAULT_MODEL_ID),
16         region_name=env.get("AWS_REGION", "us-east-1"),
17         # Local runs use the SSO profile; inside the Runtime boto3 takes the role.
18         profile_name=env.get("AWS_PROFILE") or None,
19         max_tokens=16000,
20     )
```

En local usa el perfil SSO; en el Runtime de AgentCore toma el rol de la cuenta.

Tomar las decisiones que definen la V1

Stack y harness.

- Python + LlamaIndex Workflows: pasos y eventos tipados, y cada paso se prueba por separado
- BedrockConverse: salida estructurada, tool calling y caché de prompt sin código propio
- Langfuse: el prompt versionado, las trazas de cada llamada y los experimentos
- Harness de desarrollo: propone cambios en el código, y Gastón los revisa y corre los tests

Tomar las decisiones que definen la V1

¿Con qué lo construimos?

Tomada el 28/09 · Langfuse se sumó el 1/10 · hoy se valida

DECISIÓN 7

¿Qué stack usamos para el agente?

A · Python + LlamaIndex Workflows + BedrockConverse, con Langfuse para prompts y evaluación

B · Llamadas directas al SDK, sin framework (como hoy)

C · Otro framework de agentes (Strands, LangGraph...)

PROPUESTA · A. Es el stack del template que Crafttech ya opera. El Workflow no conoce Bedrock y cada paso se prueba con dobles. B repite el código de loop, herramientas y caché que un framework ya trae. C es válido, y elegimos A por continuidad con lo que ya está probado.

decisiones/07-stack.md

Tomar las decisiones que definen la V1

¿Cuándo está bien hecho?

Esquema de Shopee y salida esperada MOCK · validar con el equipo de catálogo

- Exacto: categoría correcta, ningún atributo de más ni de menos, nada fuera de la lista del canal, sin duplicados y con los faltantes informados
- También medimos precisión y recall de atributos, valores inválidos, tokens y segundos, todo en Langfuse
- Dataset: 30 productos reales con su publicación actual. El esquema de Shopee y la salida esperada son simulados

Tomar las decisiones que definen la V1

La métrica, en código.

core/src/catalog/evaluation.py · líneas 18–32

```
18 @dataclass
19 class CaseResult:
20     category_ok: bool
21     tp: int
22     fp: int
23     fn: int
24     invalid: list[str] = field(default_factory=list)
25     duplicates: list[str] = field(default_factory=list)
26     missing_not_detected: list[str] = field(default_factory=list)
27     extra_missing: list[str] = field(default_factory=list)
28
29     @property
30     def exact(self) -> bool:
31         return (self.category_ok and self.fp == 0 and self.fn == 0 and not self.invalid
32                 and not self.duplicates and not self.missing_not_detected and not self.extra_missing)
```

La misma función mide el proceso de hoy y cada versión.

Tomar las decisiones que definen la V1

¿Cuál es el número que aceptamos?

DECISIÓN 8

¿Qué criterio de éxito acordamos para la prueba de las 16:15?

A · Cero valores inválidos, cero duplicados y todos los obligatorios informados en los 30. Los exactos se informan, pero no son condición hasta validar la salida esperada con catálogo

B · Exactos iguales o mayores al 80 %

C · Solo precisión y recall de atributos

PROPUESTA · A. Con una salida esperada mock, el exacto castiga aciertos que hoy nadie mapea: el proceso actual saca 7 de 30. Los inválidos, duplicados y obligatorios sin informar sí son errores seguros, y hoy hay 47, 5 y 2. El número final se fija en la sala.

decisiones/08-criterio-de-exito.md

Tomar las decisiones que definen la V1

¿Con qué dataset?

DECISIÓN 9

¿Sobre qué datos construimos y medimos?

A · Los 30 reales del zip, con esquema y expected mock rotulados

B · Solo los 10 casos mock de borde

C · Esperar el esquema oficial de Shopee para empezar

PROPUESTA · A. Son productos reales, con publicaciones reales y rechazos reales de Shopee. Lo simulado queda rotulado y se valida con catálogo después. Los 10 casos mock quedan como pruebas de borde.

decisiones/09-dataset.md

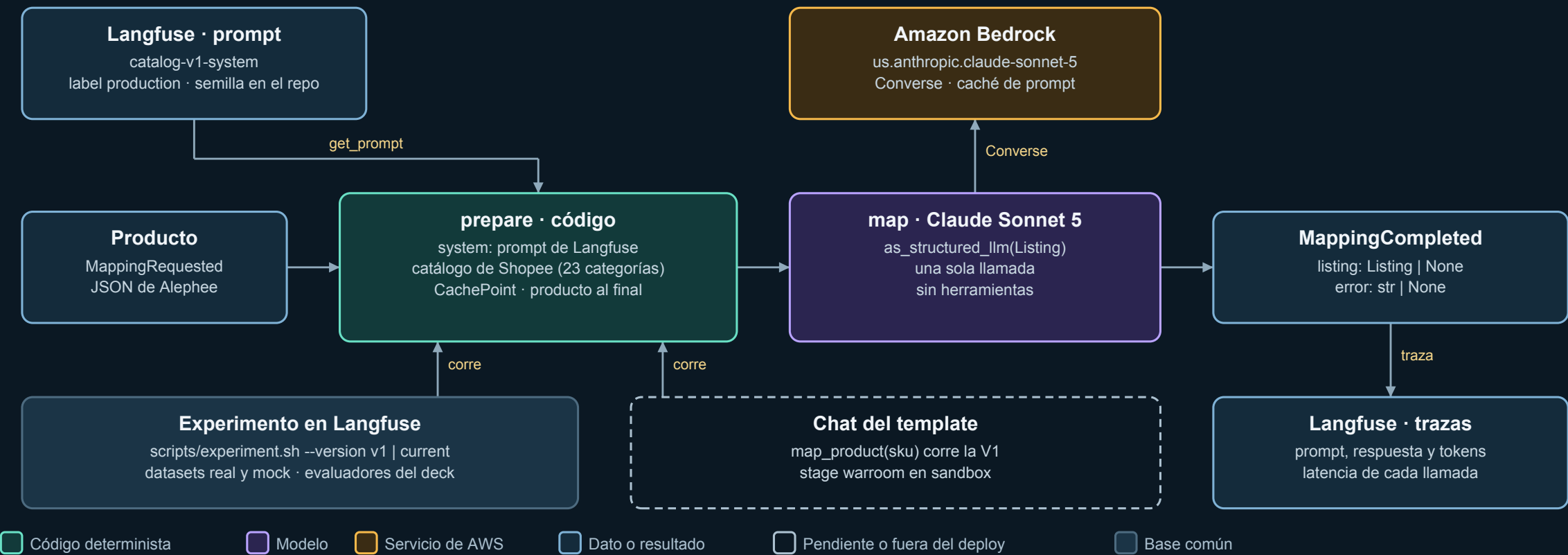
Construir, correr y leer la primera versión

V1 · el agente responde.

Una sola llamada al modelo, sin herramientas. Esperamos que falle, y esos fallos justifican lo que sigue.

Construir, correr y leer la primera versión

La V1 por dentro.



La V1 es un Workflow de dos steps. El experimento y el chat corren el mismo Workflow, y Langfuse guarda el prompt, las trazas y los resultados.

Construir, correr y leer la primera versión

El prompt de sistema.

core/src/catalog/prompts/catalog-v1-system.txt · líneas 7–18

```
7 Rules:
8 - Copy the category URN and every attribute URN exactly from the Shopee catalog. Never invent a URN.
9 - If the channel attribute has a list of values, choose the equivalent value from that list and use
10   its id and its name exactly as listed. If no value is equivalent, do not fill it.
11 - If the channel attribute is free text, use valueId "0" and the product value as it is.
12 - Never translate values: Shopee values stay in Portuguese, exactly as the catalog lists them.
13 - Never invent values. Use only data present in the product attributes.
14 - The values "-1", "N/A" or empty mean there is no data.
15 - If a mandatory channel attribute cannot be filled, add it to "missing" with the reason.
16 - If a product attribute cannot be used, add it to "rejected" with its legacyId and the reason.
17 - Each channel attribute appears at most once.
18 - If you cannot choose a category, return category null and add {"urn": "category"} to "missing".
```

Esta copia es la semilla. La versión que se usa vive en Langfuse con el label production.

Construir, correr y leer la primera versión

El prompt vive en Langfuse.

core/src/catalog/prompts.py · líneas 22–27

```
22 def get_system_prompt(name: str, client) -> SystemPrompt:
23     seed = load_seed(name)
24     if client is None:
25         return SystemPrompt(seed, name, None)
26     prompt = client.get_prompt(name, label=LABEL, fallback=seed)
27     return SystemPrompt(prompt.compile(), name, None if prompt.is_fallback else prompt.version)
```

version None quiere decir que se usó la semilla.

Construir, correr y leer la primera versión

Lo fijo primero, el producto al final.

core/src/catalog/v1.py · líneas 46–55

```
46 def build_messages(system_prompt: str, schemas: dict[str, dict], product: dict) -> list[ChatMessage]:
47     return [
48         ChatMessage(role="system", content=system_prompt),
49         ChatMessage(role="user", blocks=[
50             TextBlock(text=catalog_text(schemas)),
51             # Everything before this point is identical for every product: Bedrock caches it.
52             CachePoint(cache_control=CacheControl(type="default")),
53             TextBlock(text=product_text(product)),
54         ]),
55     ]
```

En la traza de Langfuse se ven los tokens leídos de caché.

Construir, correr y leer la primera versión

El Workflow cierra con un evento tipado.

core/src/catalog/events.py · líneas 9–21

```
9 class MappingRequested(StartEvent):
10     product: dict
11
12
13 class ContextReady(Event):
14     messages: list[ChatMessage]
15
16
17 class MappingCompleted(StopEvent):
18     """End of a mapping. No field is called `result`: it would collide with StopEvent's."""
19
20     listing: Listing | None
21     error: str | None = None
```

El batch, el chat y los tests leen fin.listing sin parsear nada.

Construir, correr y leer la primera versión

Demo · un caso real por V1.

```
scripts/experiment.sh --version v1 --data real --case error-88904447
```

- La categoría que eligió el modelo y la que dice `reference_category`
- Cada atributo: de dónde sale el valor y si está en la lista del canal
- `missing` y `rejected`: si informa lo que no pudo o completa igual
- En la traza de Langfuse: tokens de entrada, tokens leídos de caché y segundos

Respaldo · la corrida del experimento v1-real de la mañana, en Langfuse

Construir, correr y leer la primera versión

Qué esperamos que falle en la V1 y qué capa lo resuelve.

Hipótesis para confirmar con la corrida del día · salida esperada MOCK

Fallo esperado	Cómo se ve	Capa que lo resuelve
Categoría inventada cuando el producto no trae una	Caso mock 09, sin categoría de origen	V2 · la tabla de categorías
Valor fuera de la lista del canal	Score invalid_values en Langfuse	V3 · validación en código
Obligatorio sin informar en missing	Score missing_ok en Langfuse	V3 · validación en código
Unos 20.000 tokens de entrada por producto	Tokens de cada traza en Langfuse	V2 · herramientas en lugar del catálogo entero
Categoría acertada con ayuda	Elige entre 23 categorías que incluyen la correcta	V2 · la tabla de categorías

Decidir qué resuelve la tabla y conectarla

V2 · herramientas.

El agente consulta antes de decidir.

Decidir qué resuelve la tabla y conectarla

Una herramienta es una función que el modelo pide y el código ejecuta.

01 El modelo pide `buscar_categoria('urn:category:1106872')`

02 El programa ejecuta la función: consulta la tabla

03 Devuelve `{encontrada: true, urn, name}` o `{encontrada: false, motivo}`

04 El modelo sigue con ese dato, no con su memoria

Decidir qué resuelve la tabla y conectarla

¿Qué decide la tabla y qué decide el agente?

Se construye en la V2 y la V3

DECISIÓN 10

¿Qué decide la tabla y qué decide el agente?

- A · La tabla fija la categoría y los campos; el agente solo elige el valor equivalente de la lista y cubre lo que la tabla no tiene
- B · El agente puede corregir la tabla si cree que está mal
- C · Todo por tabla; sin modelo

PROPUESTA · A. La tabla manda, como se acordó el 25/08 con Juan David. Lo que la tabla no cubre, como los valores de lista o los atributos sin referencia, es lo único que decide el agente.

decisiones/10-tabla-vs-agente.md

Decidir qué resuelve la tabla y conectarla

Costo por producto, medido.

Se completa con los experimentos del día en Langfuse

Versión	Entrada sin caché	Leída de caché	Segundos
V1	.	.	.
V2	.	.	.

Decidir qué resuelve la tabla y conectarla

¿Cuánto puede costar?

Abierta: todavía no hay tope ni una medición completa

DECISIÓN 11

¿Cómo tratamos la restricción de costo del modelo?

A · Tope mensual acordado con Alephee y medición por producto: tokens de entrada, salida, caché leída y escrita

B · Sin tope: se mide después

C · El tope de hoy (USD 350) y publicar sin atributos al agotarse

PROPUESTA · A. Hoy el tope corta la calidad, que es la opción C. Con la caché de prompt y la caché por SKU (decisión 13) el costo por producto baja. El número se fija con una corrida completa medida en Langfuse.

decisiones/11-costo.md

Decidir qué resuelve la tabla y conectarla

Demo · el mismo caso por V2.

```
scripts/experiment.sh --version v2 --data real --case error-88904447
```

- Qué herramientas pidió y en qué orden, en la traza de Langfuse
- La categoría ahora sale de la tabla y no del modelo
- Código OEM: ¿sigue saliendo ABS Plastic?
- Tokens leídos de caché frente a los nuevos

Respaldo · la corrida del experimento v2-real, en Langfuse

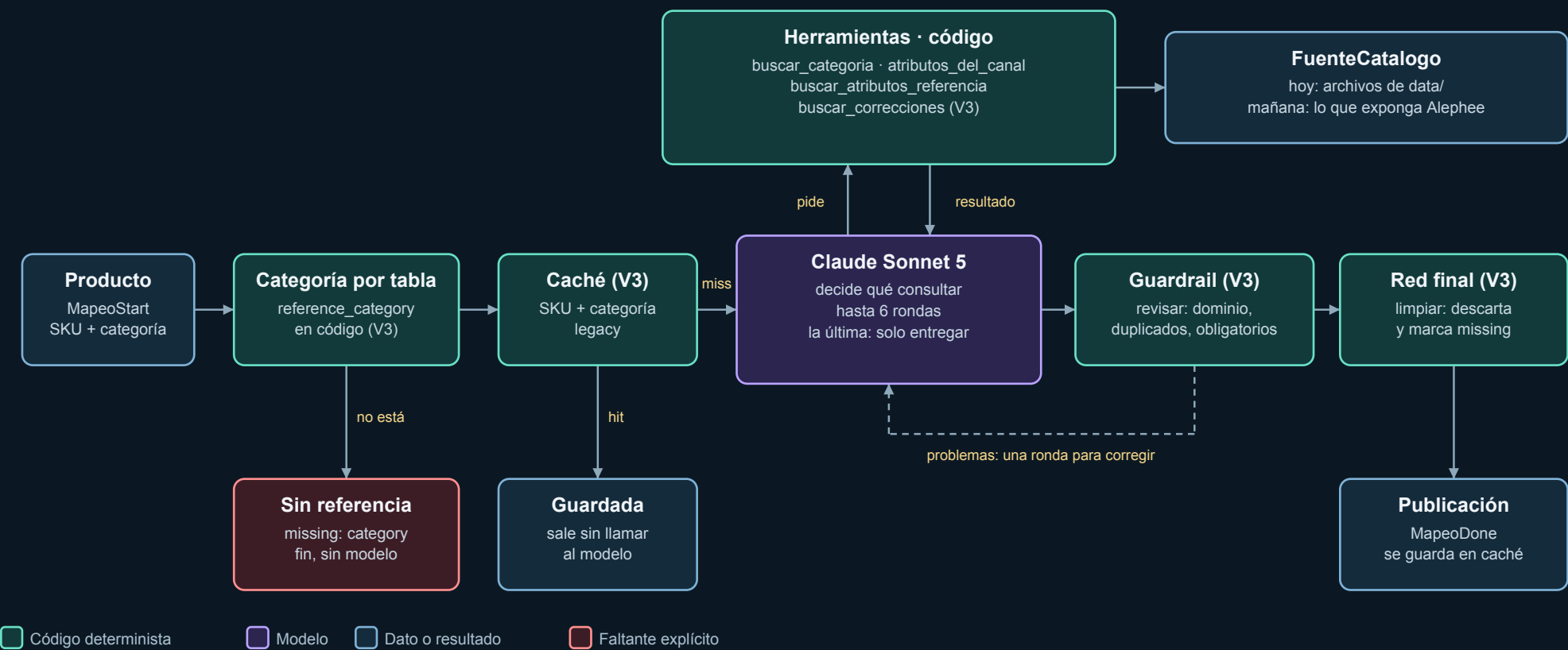
Decidir qué pasa cuando el agente no sabe

V3 · control.

Qué pasa cuando el agente no sabe.

Decidir qué pasa cuando el agente no sabe

El agente por dentro: código y modelo.



La V1 es solo la caja del modelo, con una llamada estructurada. La V2 suma las herramientas y el FunctionAgent. La V3 suma la categoría por tabla, la caché, las correcciones y las validaciones.

Decidir qué pasa cuando el agente no sabe

Guardrail: una comprobación en código.

- REVISAR | Lista los problemas en lenguaje claro y se los devuelve al agente, que tiene una ronda para corregir
- LIMPIAR | Al final descarta lo inválido con su motivo y marca los obligatorios que faltan
- NUNCA PUBLICA | Un valor fuera de la lista del canal, un -1 ni un duplicado. No verifica que el valor sea cierto: eso depende del esquema y de catálogo

Decidir qué pasa cuando el agente no sabe

¿Qué hace cuando no sabe?

La V3 se construye con la opción B · C se puede armar encima

DECISIÓN 12

¿Qué hace el agente cuando no puede completar un atributo obligatorio?

A · Publicar sin el atributo (hoy)

B · Faltante explícito con motivo: la publicación sale con missing y alguien decide

C · Bloquear la publicación hasta revisión humana

PROPUESTA · B. Reemplaza el publicar sin atributos. Quién recibe la lista missing y si bloquea la publicación es parte de la integración con la plataforma de Alephee, y queda pendiente con dueño. C es una política válida que se puede construir sobre B.

decisiones/12-cuando-no-sabe.md

Decidir qué pasa cuando el agente no sabe

Memoria: correcciones del equipo de catálogo.

- Una corrección dice: en esta categoría, este valor del producto va a este atributo con este valor del canal
- El agente la consulta como una herramienta más, y la corrección manda sobre su criterio
- En la V3 de hoy es un archivo local. En producción, una tabla de Alephee o AgentCore Memory

Decidir qué pasa cuando el agente no sabe

¿Cómo garantizamos determinismo?

La V3 se construye con la opción A · falta definir cuándo se invalida

DECISIÓN 13

¿Cómo garantizamos que el mismo SKU dé la misma salida?

A · Caché por SKU + canal; se invalida al corregir o al cambiar las tablas

B · Seed fijo (lo que se intentó hoy; los parámetros exactos siguen sin confirmar)

C · Recalcular siempre y aceptar variación

PROPUESTA · A. Un mapeo por SKU y canal. B reduce la variación pero no la elimina, y sigue pagando cada corrida. Antes de producción, la clave tiene que sumar la versión de las tablas y separar por cuenta.

decisiones/13-determinismo-y-cache.md

Decidir qué pasa cuando el agente no sabe

Demo · corregir y repetir.

```
scripts/experiment.sh --version v3 --data real --case error-88904447
```

- La consulta de correcciones aparece en la traza
- El valor corregido sale tal cual lo cargó catálogo
- rejected explica cada descarte de la validación
- Segunda corrida del mismo caso: sale de caché, sin tokens

Respaldo · la corrida del experimento v3-real, en Langfuse

Medir contra el criterio y repartir lo que sigue

La prueba.

El proceso de hoy contra V1, V2 y V3, sobre los 30, con el criterio de la decisión 8.

Medir contra el criterio y repartir lo que sigue

Resultados (se completan en vivo).

Columna Hoy: publicaciones que Alephee exportó, salida esperada MOCK. Las demás salen de los experimentos de Langfuse

Métrica	Hoy	V1	V2	V3
Casos exactos	7	.	.	.
Categoría correcta	28	.	.	.
Valores inválidos (-1 o fuera de lista)	47	.	.	.
Duplicados	5	.	.	.
Obligatorios sin informar	2	.	.	.

Medir contra el criterio y repartir lo que sigue

Cómo leer la tabla.

- Exactos bajos en todas las columnas: la salida esperada hereda las omisiones del proceso actual y castiga aciertos que hoy nadie mapea
- Inválidos y duplicados sí son errores seguros: es lo primero que tiene que bajar
- Categoría: la V1 elige entre 23 opciones con la correcta adentro, así que su número es optimista

Medir contra el criterio y repartir lo que sigue

Camino a producción.

01 Esquema oficial de Shopee y salida esperada validada con el equipo de catálogo

02 Integración: la API pública lee el producto (F8), pero escribir publicaciones y leer tablas necesita acceso interno

03 Decidir dónde corre el batch (decisión 5) y qué pasa con missing (decisión 12)

04 Medir el costo completo y fijar el tope (decisión 11), y probar Haiku 4.5 (decisión 6)

Medir contra el criterio y repartir lo que sigue

Las 13 decisiones (1 a 7).

N	Decisión	Archivo
1	Alcance: un canal, una familia, categoría + atributos	decisiones/01-alcance.md
2	Contrato: missing y rejected con motivo	decisiones/02-contrato.md
3	Tipo de aplicación: una llamada estructurada, y escalar	decisiones/03-tipo-de-aplicacion.md
4	Salida estructurada con Pydantic (Listing)	decisiones/04-salida-estructurada.md
5	Dónde corre: local hoy, chat en AgentCore, batch a medir	decisiones/05-donde-corre.md
6	Modelo: Claude Sonnet 5; Haiku 4.5 a probar	decisiones/06-modelo.md
7	Stack: Python + LlamaIndex Workflows + BedrockConverse + Langfuse	decisiones/07-stack.md

Medir contra el criterio y repartir lo que sigue

Las 13 decisiones (8 a 13).

N	Decisión	Archivo
8	Criterio de éxito: el número de la pizarra	decisiones/08-criterio-de-exito.md
9	Dataset: 30 reales con mock rotulado	decisiones/09-dataset.md
10	La tabla manda; el agente elige valores y lo no cubierto	decisiones/10-tabla-vs-agente.md
11	Costo: tope acordado y medición por producto	decisiones/11-costo.md
12	Cuando no sabe: faltante explícito, nunca inventar	decisiones/12-cuando-no-sabe.md
13	Determinismo: caché por SKU + canal con invalidación	decisiones/13-determinismo-y-cache.md

Medir contra el criterio y repartir lo que sigue

Quién hace qué, para cuándo.

Se completa en la sala.